# Teoria współbieżności

Sprawozdanie – zadania 1 i 2: wyścigi przy współdzielonym liczniku

Maksymilian Siemek | 5 października 2026

## 1. Treść zadań

**Zadanie 1.**

1. Napisać program, który uruchamia 2 wątki: jeden zwiększa wartość zmiennej całkowitej `Counter` o 1, drugi zmniejsza ją o 1. Początkowo `Counter = 0`. Jaka będzie wartość zmiennej po 10 000 operacji zwiększania i zmniejszania?
2. Na podstawie 100 wykonań programu z p. 1.1 stworzyć histogram końcowych wartości `Counter`.
3. Wprowadzić autorski mechanizm (bez mechanizmów systemowych, np. `synchronized`), który zagwarantuje przewidywalną końcową wartość `Counter`.

**Zadanie 2.** $N$ niesynchronizowanych wątków dzieli licznik (stan początkowy 0). Każdy wątek 5 razy inkrementuje licznik, a inkrementacja to sekwencja `read`, `inc`, `write`.

1. Jaka jest teoretycznie najmniejsza wartość licznika po zakończeniu wszystkich wątków i jaki przeplot do niej prowadzi?
2. Udowodnić, że będzie to zawsze najmniejsza wartość.

## 2. Argumentacja rozwiązania

Programy napisano w Javie (pakiety `zadanie1_1` … `zadanie2_2`) i uruchamiano na OpenJDK 25 (Windows).

### 2.1. Brak synchronizacji (1.1)

Obiekt `Counter` ma pole `value` oraz metody `increment()` (`value++`) i `decrement()` (`value--`). Dwa wątki wykonują po 10 000 wywołań, a wątek główny czeka na nie metodą `join()`:

**src/zadanie1_1/Main.java**

```java
package zadanie1_1;

class Counter {
    private int value = 0;

    public void increment() {
        value++;
    }

    public void decrement() {
        value--;
    }

    public int getValue() {
        return value;
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 10000; i++) {
                counter.increment();
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 10000; i++) {
                counter.decrement();
            }
```

```
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Końcowa wartość zmiennej: " + counter.getValue());
    }
}
```

Przy operacjach niepodzielnych wynik wynosiłby 0. Jednak `value++` w kodzie bajtowym to trzy kroki: `getfield` (odczyt), `iadd` (zmiana) i `putfield` (zapis). Przełączenie wątków między odczytem a zapisem powoduje utratę aktualizacji (*lost update*):

| krok | t1 (++) | t2 (--) | value |
|---|---|---|---|
| 1 | odczyt: 0 | | 0 |
| 2 | | odczyt: 0 | 0 |
| 3 | zapis: 1 | | 1 |
| 4 | | zapis: −1 | −1 |

## 2.2. Histogram (1.2)

Program `zadanie1_2` powtarza eksperyment z 1.1 100 razy (za każdym razem z nowym obiektem `Counter`). Metoda `printConsoleHistogram` dzieli przedział [min, max] wyników na 10 równych przedziałów i rysuje w konsoli liczbę trafień w każdym z nich:

**src/zadanie1_2/Main.java**

```
package zadanie1_2;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

class Counter {
    private int value = 0;

    public void increment() {
        value++;
    }

    public void decrement() {
        value--;
    }

    public int getValue() {
        return value;
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        int runs = 100;
        List<Integer> results = new ArrayList<>();

        for (int r = 0; r < runs; r++) {
            Counter counter = new Counter();

            Thread t1 = new Thread(() -> {
                for (int i = 0; i < 10000; i++) counter.increment();
            });

            Thread t2 = new Thread(() -> {
                for (int i = 0; i < 10000; i++) counter.decrement();
```

```
            });

            t1.start();
            t2.start();

            t1.join();
            t2.join();

            results.add(counter.getValue());
        }

        printConsoleHistogram(results, 10);
    }

    private static void printConsoleHistogram(List<Integer> data, int numBins) {
        int min = Collections.min(data);
        int max = Collections.max(data);

        if (min == max) {
            System.out.println("Wszystkie wykonania zwróciły wartość: " + min);
            return;
        }

        double binSize = (double) (max - min) / numBins;
        int[] bins = new int[numBins];

        for (int value : data) {
            int binIndex = (int) ((value - min) / binSize);
            if (binIndex >= numBins) {
                binIndex = numBins - 1;
            }
            bins[binIndex]++;
        }

        System.out.println("Histogram końcowych wartości zmiennej Counter (100 wykonań):\n");
        for (int i = 0; i < numBins; i++) {
            int rangeStart = (int) Math.round(min + i * binSize);
            int rangeEnd = (int) Math.round(min + (i + 1) * binSize);

            System.out.printf("[%6d, %6d) | ", rangeStart, rangeEnd);
            for (int j = 0; j < bins[i]; j++) {
                System.out.print("*");
            }
            System.out.println(" (" + bins[i] + ")");
        }
    }
}
```

## 2.3. Autorski zamek – algorytm Petersona (1.3)

Operacje na liczniku umieszczono w sekcji krytycznej chronionej własnym zamkiem `PetersonLock`, który realizuje algorytm Petersona dla dwóch wątków [1]:

**src/zadanie1_3/Main.java**

```
package zadanie1_3;

// Nasz autorski zamek oparty na Algorytmie Petersona
class PetersonLock {
    // Słowo kluczowe 'volatile' jest tu niezbędne, aby wątki widziały nawzajem
    // zmiany tych zmiennych w pamięci RAM (zapobiega to cachowaniu ich w rejestrach procesora).
    // Nie jest to mechanizm synchronizujący w sensie 'locka', lecz dyrektywa pamięciowa.
    private volatile boolean flag0 = false;
    private volatile boolean flag1 = false;
    private volatile int turn = 0;

    // Metody dla Wątku 0 (inkrementującego)
    public void lock0() {
        flag0 = true;           // Zgłaszam chęć wejścia
```

```
        turn = 1;                // Ale daję pierwszeństwo drugiemu wątkowi
        while (flag1 && turn == 1) {
            // Aktywne czekanie (busy waiting) - pętla kręci się w miejscu,
            // dopóki drugi wątek jest w sekcji krytycznej
        }
    }

    public void unlock0() {
        flag0 = false;           // Zwalniam zamek
    }

    // Metody dla Wątku 1 (dekrementującego)
    public void lock1() {
        flag1 = true;            // Zgłaszam chęć wejścia
        turn = 0;                // Ale daję pierwszeństwo pierwszemu
        while (flag0 && turn == 0) {
            // Aktywne czekanie
        }
    }

    public void unlock1() {
        flag1 = false;           // Zwalniam zamek
    }
}

class Counter {
    private int value = 0;

    public void increment() {
        value++;
    }

    public void decrement() {
        value--;
    }

    public int getValue() {
        return value;
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        PetersonLock lock = new PetersonLock();

        // Wątek 0
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 10000; i++) {
                lock.lock0();           // Zamykamy nasz autorski zamek
                counter.increment();    // Sekcja krytyczna
                lock.unlock0();         // Otwieramy zamek
            }
        });

        // Wątek 1
        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 10000; i++) {
                lock.lock1();           // Zamykamy zamek dla drugiego wątku
                counter.decrement();    // Sekcja krytyczna
                lock.unlock1();         // Otwieramy zamek
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();
```

```
        System.out.println("Końcowa wartość zmiennej: " + counter.getValue());
    }
}
```

Wątek 0 w każdej iteracji wykonuje `lock0()`, `counter.increment()`, `unlock0()`, a wątek 1 analogicznie `lock1()`, `counter.decrement()`, `unlock1()`. Flaga `flagX` oznacza chęć wejścia, a `turn` rozstrzyga konflikt: wątek czeka tylko wtedy, gdy drugi też chce wejść **i** ma pierwszeństwo.

Słowo `volatile` nie jest zamkiem. Zgodnie z modelem pamięci Javy [3] gwarantuje widoczność zapisów i zakaz przestawiania operacji na flagach. Algorytm Petersona wymaga właśnie takiej sekwencyjnej spójności. Relacja *happens-before* między zapisami i odczytami `volatile` sprawia też, że zwykłe pole `Counter.value` zapisane przed `unlock` jest widoczne dla drugiego wątku po `lock`.

## 2.4. Model i przeplot minimalny (2.1)

Każdy wątek wykonuje 5 iteracji: `READ` (pamięć $\longrightarrow$ rejestr $r$), `INC` ($r := r + 1$), `WRITE` ($r \longrightarrow$ pamięć). Najmniejszy wynik daje przeplot, w którym dwa wątki $W_1$, $W_2$ wykonują **opóźnione zapisy** starych wartości. Pozostałe wątki mogą wykonać się w całości w dowolnym momencie przed krokiem 4, bo ich praca zostanie nadpisana:

| krok | wątek | działanie | licznik |
|---|---|---|---|
| 1 | $W_2$ | iteracja 1: `READ` $\longrightarrow r_2 = 0$, wywłaszczenie | 0 |
| 2 | $W_1$ | iteracje 1–4 w całości | 4 |
| 3 | $W_3 \ldots W_N$ | wszystkie iteracje w całości | $4 + 5(N-2)$ |
| 4 | $W_2$ | `INC`, `WRITE` ($r_2 = 1$) – nadpisuje pracę wszystkich | 1 |
| 5 | $W_1$ | iteracja 5: `READ` $\longrightarrow r_1 = 1$, wywłaszczenie | 1 |
| 6 | $W_2$ | iteracje 2–5 w całości | 5 |
| 7 | $W_1$ | `INC`, `WRITE` ($r_1 = 2$) | **2** |

Rozwiązanie zapisano w postaci komentarza w pliku:

**src/zadanie2_1/Main.java**

```
package zadanie2_1;

public class Main {
    /*
     * ZADANIE 2.1 - Wyjaśnienie najmniejszej możliwej wartości licznika
     *
     * Teoretycznie najmniejsza wartość licznika po zakończeniu działania N wątków wynosi 2 (zakładając N >= 2).
     * Wynika to z faktu, że operacja inkrementacji nie jest atomowa i składa się z trzech odrębnych kroków:
     * READ (odczyt z pamięci), INC (zwiększenie o 1 w rejestrze) i WRITE (zapis z powrotem do pamięci).
     *
     * Dokładny przeplot (kolejność instrukcji) prowadzący do wyniku 2:
     *
     * 1. Wątek 2 (W2) rozpoczyna swoją 1. iterację: robi READ i odczytuje początkowe 0.
     *    Natychmiast po tym zostaje wstrzymany (wywłaszczony) przez planistę.
     * 2. Wątek 1 (W1) wykonuje pełne 4 iteracje (odczytuje, zwiększa, zapisuje). Licznik wynosi teraz 4.
     * 3. Wszystkie pozostałe wątki w systemie (W3...WN) wykonują się w całości od początku do końca,
     *    podbijając licznik do bardzo dużej wartości.
     * 4. W2 zostaje wznowiony. Kontynuuje swoją pierwszą iterację opartą na zapamiętanym wcześniej zerze:
     *    robi INC (0+1) i WRITE (zapisuje 1). Praca wszystkich innych wątków zostaje nadpisana jedynką.
     * 5. W1 rozpoczyna swoją ostatnią, 5. iterację. Robi READ i odczytuje aktualną wartość 1.
     *    Natychmiast po tym odczycie znów zostaje wstrzymany.
     * 6. W2 wznawia pracę i bez przerw wykonuje swoje pozostałe 4 iteracje.
     *    Ostatnim jego działaniem jest zapisanie w pamięci wartości 5, po czym W2 kończy życie.
     * 7. Na samym końcu budzi się wstrzymany w kroku 5. Wątek 1.
     *    Jako że pamięta w swoim rejestrze odczytaną wcześniej wartość 1, wykonuje instrukcję INC (1+1)
     *    oraz swój ostateczny WRITE, zapisując na sam koniec programu wartość 2.
     */
}
```

## 2.5. Dowód minimalności (2.2)

**Lemat.** Każdy `WRITE` zapisuje wartość $\geq 1$, więc po pierwszym zapisie licznik jest zawsze $\geq 1$.
*Dowód.* Pamięć zawiera 0 albo wartości wcześniej zapisane. Indukcyjnie: jeśli dotychczasowe zapisy były $\geq 1$, to odczyt daje $r \geq 0$, a zapis $r + 1 \geq 1$. $\square$

**Twierdzenie.** Końcowa wartość licznika jest $\geq 2$.
*Dowód.* Końcowa wartość to wartość ostatniego zapisu w wykonaniu. Wykonuje go pewien wątek $A$ w swojej 5. iteracji i zapisuje $r + 1$, gdzie $r$ odczytał `READ` tej samej iteracji. Instrukcje wątku wykonują się w kolejności programu, więc ten odczyt następuje po zapisie z 1. iteracji $A$. Z lematu $r \geq 1$, czyli wynik $\geq 2$. □

Pierwotna, opisowa wersja dowodu zapisana w kodzie:

**src/zadanie2_2/Main.java**

```
package zadanie2_2;

public class Main {
    /*
    * ZADANIE 2.2 - Dowód na to, dlaczego 2 to absolutne minimum
    *
    * Aby końcowa wartość licznika po zakończeniu całego programu wynosiła 1,
    * absolutnie ostatnia instrukcja WRITE w systemie musiałaby zapisać wartość 1.
    * To oznacza, że wątek wykonujący ten zapis musiałby w swojej ostatniej (piątej)
    * iteracji pętli odczytać z pamięci wartość 0.
    *
    * Jest to jednak logicznie niemożliwe z następujących powodów:
    *
    * 1. Żaden wątek w systemie nigdy nie zmniejsza licznika ani nie zapisuje zera.
    *    Zawsze zapisywana jest wartość o 1 większa od odczytanej (czyli co najmniej 1).
    * 2. Zanim jakikolwiek wątek dotrze do swojej ostatniej (piątej) iteracji,
    *    musiał już wykonać cztery wcześniejsze zapisy do pamięci. To gwarantuje, że
    *    początkowe zero dawno przestało istnieć.
    * 3. Najmniejszą wartością, jaka może "magicznie" pojawić się w pamięci tuż przed
    *    ostatnim odczytem, jest 1. Może ona pochodzić wyłącznie od innego wątku,
    *    który w swojej pierwszej iteracji zapamiętał początkowe 0, usnął na długi czas
    *    i dopiero teraz obudził się, by zapisać 1 (tak jak to opisano w przeplocie z 2.1).
    *
    * Wniosek:
    * Skoro najmniejszą wartością, jaką jakikolwiek wątek może odczytać (READ) na starcie
    * swojej ostatniej iteracji jest 1, to po dodaniu jedynki (INC) najmniejszą możliwą
    * do zapisania wartością (WRITE) kończącą program jest 1 + 1 = 2.
    */
}
```

## 3. Uzyskane wartości

**Zadanie 1.1** – pięć kolejnych uruchomień: **5498, 315, −333, 157, 1797**.

**Zadanie 1.2** – histogram ze 100 wykonań (rys. 1). Wyniki mieszczą się w przedziale $[-2979, 2852]$, a 84 ze 100 wykonań trafiło do przedziału zawierającego 0.




Rysunek 1: Histogram końcowych wartości `Counter` dla 100 wykonań programu `zadanie1_2`.

**Zadanie 1.3** – pięć kolejnych uruchomień z zamkiem Petersona: **0, 0, 0, 0, 0**.

**Zadanie 2** – najmniejsza możliwa wartość licznika: **2** dla $N \geq 2$ (dla $N = 1$ zawsze 5). Największa: $5N$ (wykonanie sekwencyjne).

# 4. Weryfikacja poprawności

**1.1–1.2.** Wartości różne od 0 i zmienne między uruchomieniami potwierdzają analizę z p. 2.1: `++`/`--` nie są atomowe i dochodzi do wyścigu. Szeroki rozrzut na histogramie świadczy o tym, że wynik zależy od przeplotu, a nie od błędu w logice programu. Liczba operacji obu wątków jest bowiem równa i bez wyścigu dawałaby 0.

**1.3.** Poprawność zamka wynika z trzech własności algorytmu Petersona [1, 2]:

- *Wzajemne wykluczanie.* Gdyby oba wątki były w sekcji, to `flag0 = flag1 = true`, a każdy wyszedłby z pętli dzięki `turn` na swoją korzyść. Ostatni wątek, który zapisał `turn`, ustawił je jednak na korzyść przeciwnika, więc to on musiałby czekać. Sprzeczność.
- *Brak zakleszczenia.* `turn` ma jedną wartość, więc warunek `while` jest fałszywy dla co najmniej jednego czekającego wątku.
- *Ograniczone czekanie.* Wątek ponownie wchodzący do sekcji oddaje `turn` drugiemu, więc czekający wejdzie najpóźniej po jednym wejściu przeciwnika.

Eksperymentalnie każde uruchomienie dało 0, czyli wartość zgodną z oczekiwaną.

**2.1–2.2.** Przeplot z tabeli sprawdzono krok po kroku: po każdym kroku podano stan licznika i rejestrów, a ostatni zapis wynosi 2. To pokazuje, że wartość 2 jest **osiągalna**. Twierdzenie z p. 2.5 pokazuje, że mniejszej wartości uzyskać się **nie da**, więc 2 jest dokładnym minimum.

# 5. Wnioski

- **Nieatomowość powoduje utratę aktualizacji.** `value++` to odczyt, modyfikacja i zapis. Wywłaszczenie między odczytem a zapisem powoduje nadpisanie cudzej zmiany, przez co wynik jest niedeterministyczny.
- **Krótki czas pracy wątków skupia wyniki wokół 0.** 10 000 iteracji trwa porównywalnie z uruchomieniem wątku, więc wątki często wykonują się niemal sekwencyjnie. Duże odchylenia (do ±3000) pojawiają się, gdy wątki faktycznie działają równolegle na dwóch rdzeniach.
- **Wzajemne wykluczanie przywraca determinizm.** Sekcja krytyczna chroniona zamkiem Petersona eliminuje przeplot wewnątrz `++`/`--`, więc wynik zawsze wynosi 0. Kosztem jest aktywne czekanie (zużycie CPU) i ograniczenie do dwóch wątków. Dla $N$ wątków trzeba użyć np. algorytmu piekarnianego Lamporta [2].
- **Bez `volatile` algorytm Petersona nie działa.** Optymalizacje JIT i procesora mogą przestawić zapisy flag lub zbuforować je w rejestrach, co łamie założenie o sekwencyjnej spójności pamięci.
- **W zadaniu 2 opóźniony zapis cofa licznik.** Wątek zapisujący wartość odczytaną dawno temu kasuje pracę innych. Licznika nie da się jednak cofnąć poniżej 2, bo zero znika z pamięci po pierwszym zapisie, a wątek wykonujący ostatni zapis sam wcześniej coś zapisał.

# 6. Bibliografia

[1] G. L. Peterson, *Myths About the Mutual Exclusion Problem*, Information Processing Letters 12(3), 1981, s. 115–116.
[2] M. Herlihy, N. Shavit, *The Art of Multiprocessor Programming*, Morgan Kaufmann, 2008, rozdz. 2.
[3] J. Gosling i in., *The Java Language Specification, Java SE 25 Edition*, rozdz. 17 „Threads and Locks", Oracle. https://docs.oracle.com/javase/specs/